NAME: CHERUKURI DIMPLE

REG NO: 192311315

ITA0610 - Machine Learning

Assignment Questions

Course Code & Name	ITA0610 - Machine Learning
Assignment Title	Adaptive Multi-Paradigm Learning Framework for Early Chronic-Disease Risk Prediction (Neural Networks, Genetic Algorithms & Bayesian Reasoning)
Course Outcome(s) - CO	CO3: Develop proficiency in neural network architectures, including perceptron models, multilayer perceptrons, and back propagation algorithms. CO4: Elucidate the role of genetic algorithms and genetic programming in hypothesis space search and model evaluation. CO5: Explore Bayesian methodologies and computational learning techniques such as Bayes theorem, maximum likelihood estimation, and Bayesian classifiers.
Bloom's Taxonomy Level	L5 - Evaluate; L6 - Create
SDG Mapping (SDG 1-17)	SDG 3 - Good Health and Well-being

Problem Statement

A healthcare-analytics startup is building an early-warning system to predict the risk of a chronic disease (e.g., diabetes or cardiovascular disease) from patient vitals, lifestyle, and laboratory records collected across multiple clinics. Reliable early prediction is critical for reducing preventable complications and directly supports SDG 3 - Good Health and Well-being. As the lead ML engineer, you must design, implement, and rigorously evaluate an adaptive, multi-paradigm learning framework that combines complementary learning strategies rather than relying on a single algorithm or any pre-built machine-learning library.

(a) Implement a Multilayer Perceptron entirely from first principles using only NumPy - including forward propagation, back-propagation with at least one hidden layer, and a justified activation-function choice - trained to predict disease risk from the patient dataset.

(b) Implement a Genetic Algorithm from scratch to optimise the MLP's initial weight configuration and/or key hyperparameters (number of hidden units, learning rate). Define a suitable fitness function and selection, crossover, and mutation operators, and empirically demonstrate improved convergence over random weight initialisation.

(c) Implement a Naive Bayes classifier from first principles as an independent probabilistic baseline, manually deriving the posterior-probability computation for at least one test patient record.

(d) Design a decision-fusion mechanism (e.g., a weighted scheme or Bayesian model averaging) that combines the MLP+GA prediction with the Naive Bayes prediction, and mathematically justify the fusion strategy chosen.

(e) Rigorously evaluate all three configurations (MLP alone, MLP+GA, fused model) using accuracy, precision, recall, F1-score, and ROC-AUC under k-fold cross-validation, and statistically justify whether the observed improvements are significant.

(f) Analyse the computational-learning-theoretic aspects of your solution: estimate the sample complexity required for your hypothesis space and discuss where your dataset size sits relative to PAC-learning bounds.

(g) Critically discuss the ethical, fairness, and privacy implications of deploying such a system in real clinical settings, and explain how your design choices support SDG 3 targets on reducing premature mortality from non-communicable diseases.

Constraints: All core algorithms (multilayer perceptron, back-propagation, genetic algorithm, Naive Bayes) must be implemented from first principles using only NumPy/Pandas; scikit-learn, TensorFlow, PyTorch, or any other pre-built machine-learning or optimisation library may be used only to independently cross-check your own results, never to implement the core logic. Use a real, publicly available clinical dataset (for example, the Pima Indians Diabetes Dataset or the UCI Heart Disease Dataset) containing at least 500 records. The pipeline must handle missing or noisy data and class imbalance, and must run end-to-end without manual intervention.

Adaptive Multi-Paradigm Learning Framework for Early Chronic-Disease Risk Prediction

1. Introduction

Early prediction of chronic diseases such as diabetes is an important healthcare application of machine learning. Detecting disease risk at an early stage can help healthcare professionals provide timely treatment, lifestyle recommendations, and continuous monitoring.

This project proposes an **Adaptive Multi-Paradigm Learning Framework** that combines three complementary approaches:

1. **Multilayer Perceptron (MLP)** – for learning nonlinear relationships between patient attributes and disease risk.
2. **Genetic Algorithm (GA)** – for optimizing the initial configuration of the neural network.
3. **Naive Bayes (NB)** – as an independent probabilistic classifier.
4. **Decision Fusion** – to combine the predictions of the optimized MLP and Naive Bayes models.

The complete system is implemented from first principles using **NumPy and Pandas**, without using pre-built machine-learning algorithms for the core implementation.

The project uses the **Pima Indians Diabetes Dataset**, a publicly available clinical dataset containing 768 patient records and eight input attributes.

---

2. Dataset Description

2.1 Dataset Used

Dataset: Pima Indians Diabetes Dataset

Number of records: 768

Number of input features: 8

Target variable: Outcome

Problem type: Binary classification

The objective is to predict whether a patient has diabetes based on medical and demographic attributes.

The dataset contains 500 records with Outcome = 0 and 268 records with Outcome = 1. Therefore, the dataset has moderate class imbalance, which must be considered during preprocessing and evaluation.

2.2 Dataset Attributes

Feature	Description
Pregnancies	Number of times the patient has been pregnant
Glucose	Plasma glucose concentration
BloodPressure	Diastolic blood pressure
SkinThickness	Triceps skin-fold thickness
Insulin	2-hour serum insulin level
BMI	Body Mass Index
DiabetesPedigreeFunction	Diabetes pedigree/family-history risk measure
Age	Age of the patient
Outcome	Target variable: 0 = No Diabetes, 1 = Diabetes

Sample Dataset

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1

---

3. Proposed System Architecture

The proposed system follows the pipeline:

Patient Dataset → Data Preprocessing → Train/Test Split → MLP → Genetic Algorithm Optimization → Naive Bayes → Decision Fusion → Performance Evaluation

Major Modules

1. Dataset Collection
2. Data Preprocessing
3. MLP Implementation
4. Backpropagation
5. Genetic Algorithm Optimization
6. Naive Bayes Implementation
7. Decision Fusion
8. Cross-Validation
9. Performance Evaluation
10. Statistical Significance Testing
11. Learning-Theory Analysis
12. Ethical and SDG 3 Analysis

---

4. Data Preprocessing

Healthcare datasets may contain missing, noisy, or invalid measurements. Therefore, preprocessing is performed before model training.

4.1 Handling Missing Values

In the Pima dataset, some medical measurements contain zero values that are medically unrealistic for certain variables.

The following columns are checked for zero values:

- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI

These zero values are treated as missing values.

The missing values are replaced using the **median** of the corresponding feature.

Median imputation is preferred because it is less sensitive to extreme values than mean imputation.

4.2 Duplicate Removal

Duplicate patient records are checked and removed to avoid giving excessive importance to repeated observations.

4.3 Noise Handling

Extreme observations are detected using percentile-based clipping. Values below the 1st percentile or above the 99th percentile can be clipped to reduce the effect of extreme noise.

4.4 Feature Scaling

The input features have different numerical ranges. Therefore, standardization is applied:

Xscaled=X−μσX_{scaled}=\frac{X-\mu}{\sigma}

where:

- XX = original feature value
- μ\mu = feature mean
- σ\sigma = feature standard deviation

Scaling helps the neural network converge more effectively.

4.5 Class Imbalance

The dataset contains:

- Class 0: 500 samples
- Class 1: 268 samples

To prevent the majority class from dominating the learning process, stratified cross-validation and class-aware evaluation metrics are used.

Accuracy alone is therefore not considered sufficient.

5. Multilayer Perceptron

5.1 Objective

The Multilayer Perceptron is implemented completely from first principles using NumPy.

The MLP learns a nonlinear mapping:

X→Hidden Layer→OutputX \rightarrow Hidden\ Layer \rightarrow Output

The network contains:

- Input layer: 8 neurons
- Hidden layer: configurable number of neurons
- Output layer: 1 neuron

---

6. Activation Function

The hidden layer uses the **ReLU activation function**:

ReLU(x)=max(0,x)ReLU(x)=max(0,x)

Its derivative is:

ReLU′(x)={1x>00x≤0ReLU'(x)= \begin{cases} 1 & x>0\\ 0 & x\leq0 \end{cases}

ReLU is selected because it is computationally simple and helps reduce the vanishing-gradient problem compared with sigmoid activation in hidden layers.

The output layer uses the **Sigmoid activation function**:

σ(x)=11+e−x\sigma(x)=\frac{1}{1+e^{-x}}

The sigmoid function produces a probability between 0 and 1, making it appropriate for binary diabetes prediction.

---

7. Forward Propagation

For an input vector XX, the hidden-layer weighted sum is:

Z1=XW1+b1Z_1=XW_1+b_1

The hidden activation is:

A1=ReLU(Z1)A_1=ReLU(Z_1)

The output-layer weighted sum is:

Z2=A1W2+b2Z_2=A_1W_2+b_2

The final prediction probability is:

y^=σ(Z2)\hat{y}=\sigma(Z_2)

A threshold of 0.5 is used:

Prediction={1y^≥0.50y^<0.5Prediction= \begin{cases} 1 & \hat{y}\geq0.5\\ 0 & \hat{y}<0.5 \end{cases}

---

8. Loss Function

Binary Cross-Entropy is used as the loss function:

L=−1N∑i=1N[yilog⁡(yi^)+(1−yi)log⁡(1−yi^)]L=-\frac{1}{N}\sum_{i=1}^{N} [y_i\log(\hat{y_i})+(1-y_i)\log(1-\hat{y_i})]

where:

- yiy_i is the actual label
- yi^\hat{y_i} is the predicted probability
- NN is the number of training samples

The objective of training is to minimize this loss.

---

9. Backpropagation

Backpropagation calculates the gradient of the loss with respect to each network parameter.

For the output layer:

δ2=y^−y\delta_2=\hat{y}-y

The gradient of the output weights is:

∂L∂W2=A1Tδ2\frac{\partial L}{\partial W_2} =A_1^T\delta_2

The gradient of the output bias is:

∂L∂b2=∑δ2\frac{\partial L}{\partial b_2} =\sum\delta_2

The hidden-layer error is:

δ1=(δ2W2T)⊙ReLU′(Z1)\delta_1=(\delta_2W_2^T)\odot ReLU'(Z_1)

The hidden-layer weight gradient is:

∂L∂W1=XTδ1\frac{\partial L}{\partial W_1} =X^T\delta_1

The hidden-layer bias gradient is:

$$\frac{\partial L}{\partial b_1} = \sum \delta_1$$

The parameters are updated using gradient descent:

$$W = W - \eta \frac{\partial L}{\partial W}$$

where η is the learning rate.

---

10. Genetic Algorithm

10.1 Objective

The Genetic Algorithm is used to optimize the initial configuration of the MLP.

Instead of randomly selecting neural-network weights, GA searches for a better initial weight configuration.

The GA consists of:

1. Population initialization
2. Fitness evaluation
3. Selection
4. Crossover
5. Mutation
6. Replacement
7. Multiple generations

---

11. GA Representation

Each chromosome represents a complete set of MLP parameters:

$$Chromosome = [W_1, b_1, W_2, b_2]$$

The parameters are flattened into a one-dimensional vector for genetic operations.

---

12. Fitness Function

The fitness function measures the quality of a chromosome.

A suitable fitness function is:

$$Fitness = Validation\ Accuracy$$

Alternatively, validation loss can be minimized:

$$Fitness = -Validation\ Loss$$

Higher fitness represents a better initial configuration.

The GA aims to find:

W∗=arg⁡max⁡WFitness(W)W^*=\arg\max_W Fitness(W)

---

13. Selection

Tournament selection is used.

A small number of chromosomes are randomly selected from the population. The chromosome having the highest fitness becomes a parent.

This process is repeated to select the required parents.

---

14. Crossover

Two parent chromosomes are combined to produce offspring.

For a single crossover point:

Child=[Parent1[:k],Parent2[k:]]Child=[Parent_1[:k],Parent_2[k:]]

where kk is a randomly selected crossover position.

Crossover allows useful parameter configurations from different parents to be combined.

---

15. Mutation

Random changes are introduced to maintain population diversity.

A Gaussian mutation can be applied:

x′=x+ϵx'=x+\epsilon

where:

ϵ∼N(0,σ2)\epsilon\sim N(0,\sigma^2)

Mutation helps the algorithm escape local solutions.

---

16. GA Optimization Process

The process is:

Random Population → Fitness Evaluation → Selection → Crossover → Mutation → New Population → Repeat

The best chromosome obtained after several generations is used to initialize the MLP.

---

17. Empirical Comparison

Two MLP configurations are compared:

MLP with Random Initialization

Weights are initialized randomly and training starts directly.

MLP with GA Initialization

Weights are first optimized using the Genetic Algorithm and then used to initialize the MLP.

The following values are recorded:

Configuration	Initial Loss	Final Loss	Accuracy	F1 Score
MLP Random	To be obtained from execution	To be obtained	To be obtained	To be obtained
MLP + GA	To be obtained from execution	To be obtained	To be obtained	To be obtained

Important: Do not invent these numerical results in the Word document. After running the Colab code, paste the actual values generated by your experiment.

---

18. Naive Bayes Classifier

Naive Bayes is implemented independently from first principles.

For a patient record XX, Bayes' theorem is:

P(C|X)=P(X|C)P(C)P(X)P(C|X)=\frac{P(X|C)P(C)}{P(X)}

For classification, the denominator is common to all classes, therefore:

P(C|X)∝P(C)∏j=1dP(Xj|C)P(C|X)\propto P(C)\prod_{j=1}^{d}P(X_j|C)

where:

- CC = class
- XjX_j = feature jj
- dd = number of features

---

19. Gaussian Naive Bayes

Since most medical attributes are continuous, Gaussian probability distributions are used.

For feature xx and class cc:

P(x|c)=12πσc2e−(x−μc)22σc2P(x|c)= \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{-\frac{(x-\mu_c)^2}{2\sigma_c^2}}

where:

- μc\mu_c = class-specific mean
- σc2\sigma_c^2 = class-specific variance

The posterior score becomes:

Score(c)=P(c)∏j=1dP(xj|c)Score(c)=P(c)\prod_{j=1}^{d}P(x_j|c)

The class with the larger posterior probability is selected.

---

20. Manual Naive Bayes Posterior Derivation

For one test patient:

X=[x1,x2,…,x8]X=[x_1,x_2,\ldots,x_8]

The probability of no diabetes is:

P(C=0|X)∝P(C=0)∏j=18P(xj|C=0)P(C=0|X) \propto P(C=0) \prod_{j=1}^{8}P(x_j|C=0)

The probability of diabetes is:

P(C=1|X)∝P(C=1)∏j=18P(xj|C=1)P(C=1|X) \propto P(C=1) \prod_{j=1}^{8}P(x_j|C=1)

After calculating the Gaussian probability for every feature, the two scores are normalized:

P(C=0|X)=Score0Score0+Score1P(C=0|X)= \frac{Score_0}{Score_0+Score_1} P(C=1|X)=Score1Score0+Score1P(C=1|X)= \frac{Score_1}{Score_0+Score_1}

The class with the larger posterior probability becomes the final Naive Bayes prediction.

Example Format for Report

Use one actual test patient generated by your program and fill the following table with the **actual values from your execution**:

Quantity	Value
P(C=0)P(C=0)	Actual result
P(C=1)P(C=1)	Actual result
(P(X	C=0))
(P(X	C=1))
Posterior (P(C=0	X))

Quantity	Value
Posterior (P(C=1 X))	
Final Prediction	0 or 1

---

21. Decision Fusion

The final system combines the MLP+GA prediction with the Naive Bayes probability.

Let:

PMLP=P(Y=1|X)MLPP_{MLP}=P(Y=1|X)_{MLP}

and

PNB=P(Y=1|X)NBP_{NB}=P(Y=1|X)_{NB}

A weighted fusion approach is used:

PFusion=αPMLP+(1−α)PNBP_{Fusion} = \alpha P_{MLP}+ (1-\alpha)P_{NB}

where:

0≤α≤10\leq\alpha\leq1

For example, if equal importance is assigned:

α=0.5\alpha=0.5

then:

PFusion=0.5PMLP+0.5PNBP_{Fusion} = 0.5P_{MLP}+0.5P_{NB}

The final class is:

Prediction={1PFusion≥0.50PFusion<0.5Prediction= \begin{cases} 1 & P_{Fusion}\geq0.5\\ 0 & P_{Fusion}<0.5 \end{cases}

The fusion mechanism combines the nonlinear learning ability of the neural network with the probabilistic reasoning of Naive Bayes.

---

22. Mathematical Justification of Fusion

The weighted fusion model can be interpreted as a convex combination of two probability estimates.

Because:

0≤PMLP≤10\leq P_{MLP}\leq1

and:

$0 \leq P_{NB} \leq 1$

the fused probability also satisfies:

$0 \leq P_{Fusion} \leq 1$

The fusion therefore provides a stable probability estimate while allowing both models to contribute to the final decision.

The weight α can also be selected using validation performance. A higher value can be assigned to the model with better validation performance.

---

23. K-Fold Cross-Validation

To obtain a reliable estimate of model performance, **k-fold cross-validation** is used.

For example:

$k = 5$

The dataset is divided into five approximately equal folds.

For every iteration:

- Four folds are used for training.
- One fold is used for testing.
- The process is repeated five times.
- The final performance is calculated as the average across all folds.

Stratification is used so that each fold maintains a similar class distribution.

---

24. Models Evaluated

Three configurations are evaluated:

Configuration 1: MLP
MLP
Configuration 2: MLP + Genetic Algorithm
$GA \rightarrow MLP$
Configuration 3: Fusion
$MLP + GA + NB$

---

25. Evaluation Metrics

25.1 Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy measures the proportion of correctly classified patients.

25.2 Precision

$$Precision=\frac{TP}{TP+FP}$$

Precision measures how many patients predicted as diabetic were actually diabetic.

25.3 Recall

$$Recall=\frac{TP}{TP+FN}$$

Recall is particularly important in healthcare because false negatives can result in missed disease cases.

25.4 F1 Score

$$F1=2\frac{Precision\times Recall}{Precision+Recall}$$

F1 provides a balance between precision and recall.

25.5 ROC-AUC

ROC-AUC measures the ability of the model to distinguish between positive and negative classes across different classification thresholds.

A value closer to 1 indicates stronger discrimination.

26. Evaluation Results Table

After running the program, the actual results should be inserted into the following table:

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
MLP	___	___	___	___	___
MLP + GA	___	___	___	___	___
MLP + GA + NB Fusion	___	___	___	___	___

Cross-Validation Summary

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean
MLP	___	___	___	___	___	___
MLP + GA	___	___	___	___	___	___
Fusion	___	___	___	___	___	___

27. Statistical Significance

Simply observing a higher accuracy does not prove that one model is statistically better.

Therefore, the fold-wise performance values of the models are compared statistically.

A paired statistical test can be applied to the cross-validation results.

The hypotheses are:

Null Hypothesis H0H_0

There is no statistically significant difference between the performance of the two models.

Alternative Hypothesis H1H_1

There is a statistically significant difference between the performance of the two models.

Using significance level:

$\alpha=0.05\alpha=0.05$

If:

$p<0.05p<0.05$

the null hypothesis is rejected and the improvement is considered statistically significant.

If:

$p\geq0.05p\geq0.05$

there is insufficient evidence to claim a statistically significant improvement.

---

28. Computational Learning Theory

The dataset contains:

$N=768N=768$

records and:

$d=8d=8$

input features.

The MLP has a finite number of trainable parameters. For a network with hh hidden neurons:

$W1:8hW_1:8h\ b1:hb_1:h\ W2:hW_2:h\ b2:1b_2:1$

Therefore, the total number of parameters is:

$P=8h+h+h+1P=8h+h+h+1\ P=10h+1P=10h+1$

For example, if:

$h=16h=16$

then:

P=161P=161

This gives a relatively small hypothesis space compared with very large neural networks.

---

29. PAC-Learning Discussion

PAC stands for **Probably Approximately Correct** learning.

A simplified generalization relationship can be expressed as:

m=O(VC(H)+ln⁡(1/δ)ϵ)m=O\left(\frac{VC(H)+\ln(1/\delta)} {\epsilon} \right)

where:

- mm = required number of training samples
- VC(H)VC(H) = VC dimension of the hypothesis class
- ϵ\epsilon = acceptable generalization error
- δ\delta = probability of failure

For neural networks, the exact VC dimension depends on architecture and parameterization.

The Pima dataset contains only 768 records. Therefore, it is large enough for demonstrating the proposed framework but relatively small compared with datasets typically desirable for robust clinical deployment.

Consequently, cross-validation, regularization, preprocessing, and careful statistical evaluation are important to reduce overfitting.

The experimental results should therefore be interpreted as research/educational evidence rather than proof of clinical effectiveness.

---

30. Ethical Considerations

Healthcare prediction systems involve highly sensitive patient information. Therefore, ethical considerations must be addressed before deployment.

30.1 Privacy

Patient information should be anonymized and protected from unauthorized access.

The system should follow applicable data-protection and healthcare regulations.

30.2 Fairness

The model should be evaluated for performance differences across demographic groups where appropriate data are available.

A model that performs well overall may still perform poorly for a particular subgroup.

30.3 False Negatives

A false negative occurs when the system predicts that a patient is not at risk when the patient actually has the disease.

In healthcare, false negatives can be particularly harmful because they may delay further medical examination.

Therefore, recall and sensitivity should be carefully monitored.

30.4 Explainability

Healthcare professionals should be able to understand the factors contributing to a prediction.

The proposed framework should be treated as a **decision-support system**, not as an autonomous replacement for medical professionals.

30.5 Human Oversight

Final clinical decisions should remain with qualified healthcare professionals.

The model should support doctors rather than independently diagnose patients.

---

31. SDG 3 – Good Health and Well-being

This project directly supports **Sustainable Development Goal 3: Good Health and Well-being**.

Early identification of diabetes risk can help support:

- Early screening
- Preventive healthcare
- Lifestyle intervention
- Continuous monitoring
- Reduction of complications
- Improved healthcare decision support

The project is particularly relevant to SDG 3's focus on reducing premature mortality from non-communicable diseases through prevention and treatment.

---

32. Advantages of the Proposed Framework

1. MLP learns nonlinear relationships between patient features.

2. Genetic Algorithm improves the initial configuration of the MLP.
3. Naive Bayes provides an independent probabilistic perspective.
4. Decision fusion combines complementary model predictions.
5. Algorithms are implemented from first principles.
6. Missing and noisy data are handled during preprocessing.
7. Class imbalance is considered during evaluation.
8. Multiple evaluation metrics are used.
9. Cross-validation provides more reliable performance estimates.
10. Statistical testing is used to evaluate observed improvements.

---

33. Limitations

1. The Pima dataset contains only 768 records.
2. The dataset represents a specific population and may not generalize to all populations.
3. Some measurements contain zero values that may represent missing information.
4. Clinical deployment would require external validation on larger and more diverse datasets.
5. Model predictions should not be considered a medical diagnosis.
6. Genetic Algorithm optimization increases computational cost.
7. Cross-validation performance may differ from real-world clinical performance.

---

34. Expected Outcome

The proposed framework is expected to demonstrate that combining multiple learning paradigms can provide a more robust prediction system than relying on a single model.

The expected comparison is:

MLP→MLP+GA→MLP+GA+NB FusionMLP \rightarrow MLP+GA \rightarrow MLP+GA+NB\ Fusion

The experimental results will determine whether GA optimization improves MLP convergence and whether decision fusion provides better predictive performance than the individual models.

Actual numerical performance values will be obtained by executing the implementation and will be reported in the final results table.

---

35. Conclusion

The project develops an adaptive multi-paradigm framework for early diabetes-risk prediction using a real clinical dataset. A Multilayer Perceptron is implemented from first principles using NumPy, including forward propagation and backpropagation. A Genetic Algorithm is then used to optimize the initial neural-network configuration. Naive Bayes is independently implemented to provide probabilistic baseline predictions.

The predictions from the optimized MLP and Naive Bayes are combined using a mathematically defined weighted decision-fusion strategy. The three configurations are evaluated using k-fold cross-validation, accuracy, precision, recall, F1-score, ROC-AUC, and statistical significance testing.

The project also considers sample complexity, PAC-learning concepts, healthcare privacy, fairness, human oversight, and SDG 3. The resulting framework demonstrates how complementary machine-learning paradigms can be integrated into an end-to-end healthcare analytics system while maintaining transparency about its limitations.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Neural Network Design & Back-Propagation Implementation (CO3)	20	MLP and back-propagation implemented fully from first principles with correct gradient computation and strong predictive performance.	Correct MLP/back-propagation with minor inefficiencies or tuning gaps.	Basic MLP works but back-propagation has notable errors or convergence issues.	MLP/back-propagation missing, incorrect, or not implemented from scratch.
Genetic Algorithm Design & Optimisation (CO4)	20	Well-designed fitness function, selection, crossover, and mutation; clear, quantified improvement over random initialisation.	GA implemented correctly with reasonable operators; improvement shown but weakly quantified.	GA implemented but operators are shallow or improvement is marginal/unverified.	GA missing, non-functional, or not integrated with the MLP.
Bayesian Reasoning & Naive Bayes Derivation (CO5)	15	Naive Bayes implemented from scratch with a fully correct, clearly shown manual posterior derivation.	Correct implementation; derivation mostly correct with minor gaps.	Implementation works but derivation is incomplete or partly incorrect.	Naive Bayes missing, incorrect, or derivation absent.
Model Fusion, Evaluation & Statistical Rigor	20	Fusion mechanism is mathematically justified; thorough k-fold evaluation across all metrics with sound statistical significance testing.	Fusion and evaluation mostly complete; minor gaps in statistical justification.	Fusion or evaluation attempted but incomplete or superficial.	No meaningful fusion strategy or evaluation performed.
Computational Learning Theory Analysis	10	Sample-complexity estimate and PAC-bound discussion are correct, well-reasoned, and tied to the actual dataset.	Reasonable estimate/discussion with minor conceptual gaps.	Attempted but with significant conceptual errors.	Not attempted or fundamentally incorrect.
Ethical Reflection & SDG 3 Relevance	5	Thoughtful, specific discussion of ethics/privacy/fairness clearly linked to SDG 3 targets.	Reasonable discussion with adequate SDG linkage.	Generic reflection with weak SDG linkage.	No genuine ethical or SDG reflection provided.
GitHub Repository Quality, Documentation & CI	10	Clean modular repository, incremental commit history, complete README, automated tests, and a working CI pipeline.	Well-organised repository with most of the above present.	Repository present but poorly organised or missing tests/CI.	Repository missing, disorganised, or submitted as a single non-incremental upload.

Deliverables

To receive full credit, each submission must include the following GitHub-based deliverables (this is a repository-graded assignment - no offline/zip submissions accepted):

1. A complete, modular GitHub repository containing all source code (MLP, genetic algorithm, Naive Bayes, and fusion logic) organised into clearly separated modules/packages, built up through an incremental commit history that shows genuine development progress (a single 'final upload' commit is not acceptable).
2. A comprehensive README.md in the repository documenting the system architecture, setup and run instructions, the dataset source and preprocessing steps, and exact steps to reproduce every reported result.
3. Automated unit tests (e.g., using pytest) covering the core algorithmic components (forward/backward pass, GA operators, Naive Bayes posterior computation), committed to the repository.
4. A technical report (Markdown or PDF, committed to the repository) containing all mathematical derivations, the fitness-function design, evaluation metrics, cross-validation results, and the statistical-significance analysis.
5. An experiment-tracking folder (e.g., /results) in the repository containing logs, plots, and result tables for every configuration tested, so results are fully auditable from the repository alone.
6. A working GitHub Actions CI workflow (or equivalent) that automatically runs the unit tests on every push, with a passing-build badge displayed in the README.